EX:8

Image Generation Using Variational Autoencoder (VAE)

Problem Statement :

Implement a variational Autoencoder (VAE) to generate new images from a given dataset. Train the ~~Dataset~~ model to learn the latent representation of images and generate the new samples from the learned distribution.

Objectives :

* Understand the concept of generative models using latent space representations.
* Implement encoder - decoder architecture with reparameterization.
* Train a VAE on celebA and generate new human face image.
* Visualize generated samples from the latent space.

Implementation Steps :-

Step 1 : Import Required Libraries

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, Subset
```

Step2 : Configure parameters and load Dataset

```
device = torch.device('cuda' if torch.cuda.is_available()
                      else 'cpu')
image_size = 64
batch_size = 128
latent_epochs = 100
num_epochs = 5
learning_rate = 1e-3
transform = transforms.compose([
    transform.CenterCrop(178),
    transform.Resize(image_size),
    transform.ToTensor(),
    transforms.Normalize((0.5,0.5,0.5),(0.5,0.5,0.5))
])
dataset = datasets.celebA(root='data', split='train', download
                          =True, transform=transform)
dataset = subset(dataset, range(500))
dataloader = DataLoader(dataset, batch_size=batch_size,
                        shuffle=True)
```

Step3: Define Encoder Network

```
Class Encoder(nn.Module):
  def __init__(self, latent_dim):
    super(Encoder, self).__init__()
    self.conv = nn.Sequential(
      nn.Conv2d(3,64,4,2,1), nn.ReLU(),
      nn.Conv2d(64,128,4,2,1), nn.BatchNorm2d(128),
                                nn.ReLU(),
      nn.Conv2d(256,512,4,2,1), nn.BatchNorm2d(512),
                                nn.ReLU()
    )
    self.fc_mu = nn.Linear(512*4*4, latent_dim)
    self.fc_logvar = nn.Linear(512*4*4, latent_dim)
```

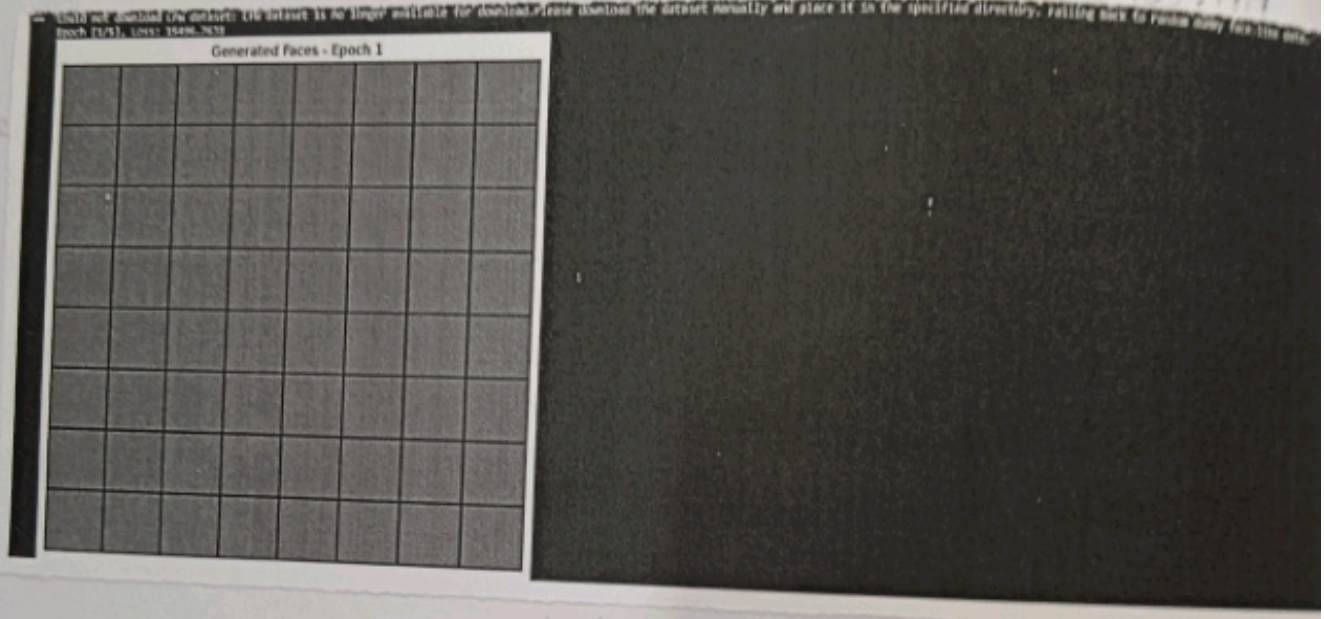
```
        x = x.view(x.size(0), -1)
        return self.fc_mu(x), self.fc_logvar(x)
```

Step 4: Define Decoder Network

```
class Decoder(nn.Module):
    def __init__(self, latent_dim):
        super(Decoder, self).__init__()
        self.fc = nn.Linear(latent_dim, 512*4*4)
        self.deconv = nn.Sequential(
            nn.ConvTranspose2d(512, 256, 4, 2, 1), nn.BatchNorm2d(256),
                                                   nn.ReLU(),
            nn.ConvTranspose2d(256, 128, 4, 2, 1), nn.BatchNorm2d(128),
                                                   nn.ReLU(),
            nn.ConvTranspose2d(64, 3, 4, 2, 1), nn.Tanh()
        )
    def forward(self, z):
        x = self.fc(z)
        x = x.view(x.size(0), 512, 4, 4)
        return self.deconv(x)
```

Step 5: Define the VAE Model

```
class VAE(nn.Module):
    def __init__(self, latent_dim):
        super(VAE, self).__init__()
        self.encoder = Encoder(latent_dim)
        self.decoder = Decoder(latent_dim)
    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5*logvar)
        eps = torch.randn_like(std)
        return mu + eps*std
    def forward(self, x):
        mu, logvar = self.encoder(x)
        z = self.reparameterize(mu, logvar)
```


Generated Faces - Epoch 1

Step 6: Define Loss function

```
def Vae_loss(recon_x, x, mu, logvar):
    recon_loss = F.mse_loss(recon_x, x, reduction='Sum')
    KL_div = -0.5 * torch.sum(1+ logvar - mu.pow(2) - logvar.exp())
    return recon_loss + KL_div
```

Step 7: Train the Model and Generate Images

```
model = VAE(latent_dim).to(device)
optimizer = torch.optim.Adam(model.parameters(), lr = learning_rate)
for epoch in range(num_epochs):
    model.train()
    total_loss = 0
    total_loss += loss.item()
    print(f"Epoch [{epoch+1}/{num_epoch}], {total_loss/len(dataloader.dataset):.4f}")
    Sample_images = model.decoder(z).cpu()*0.5 + 0.5
    grid = torchvision.utils.make_grid(Sample_image, nrow=8)
    plt.imshow(grid.permute(1,2,0))
    plt.axis('off')
    plt.title(f"Generated Faces - Epoch {epoch+1}")
    plt.show()
```

RESULT:

The VAE learned facial feature representations and successfully generated realistic synthetic faces from random latent space samples.